Mechanical Systems and Signal Processing 246 (2026) 113917

Contents lists available at ScienceDirect

Mechanical Systems and Signal Processing

journal homepage: www.elsevier.com/locate/ymssp

Full Length Article

Federated Open-Set Fault Diagnosis for Unknown Bearing Fault Detection

Danya Xu [a], Mingwei Jia [b], Tao Chen [c], Yi Liu [b], Dongyue Chen [d], Tianyou Chai [a], Tao Yang [a,*]

[a] State Key Laboratory of Synthetical Automation for Process Industries, Northeastern University, Shenyang 110819, China
[b] Institute of Process Equipment and Control Engineering, Zhejiang University of Technology, Hangzhou 310023, China
[c] Department of Chemical and Process Engineering, University of Surrey, Surrey GU2 7XH, UK
[d] College of Information Science and Engineering, Northeastern University, Shenyang 110819, China

ARTICLE INFO

Communicated by X. SI

Keywords:
Fault diagnosis
Federated learning
Open-set detection

ABSTRACT

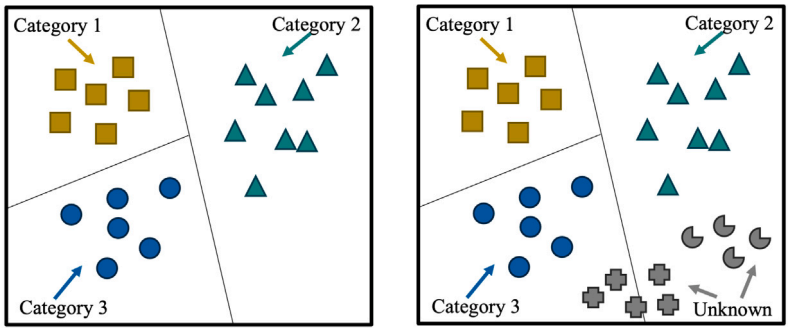
Deep learning improves the accuracy of fault diagnosis. In traditional centralized training approaches, it often requires centralizing a large amount of labeled data across multiple clients, which is impeded by privacy and security constraints. Federated learning addresses this challenge by enabling collaborative model training without sharing raw data. However, most existing federated fault diagnosis methods focus on the closed-set assumption and cannot handle unseen faults that may occur in industrial environments. To this end, we propose a Federated Open-Set Fault Diagnosis (FOSFD) method. Specifically, during the training phase, the proposed method enhances the decision boundaries of the global model by generating augmented features locally using MixUp-based data augmentation strategy and aligning prototypes of known categories through the central server. During the testing phase, anomaly scores based on distances between testing samples' feature representations and prototypes are computed to identify known or unknown faults. Experiments on Case Western Reserve University and Paderborn University bearing fault diagnosis datasets demonstrate that our method achieves state-of-the-art performance in terms of accuracy (Acc), area under the curve (AUC), and open set classification rate (OSCR), thereby delivering superior unknown-fault detection.

1. Introduction

Fault diagnosis is essential for ensuring operating safety and reliable production in industrial process [1,2], as undetected faults can lead to equipment failure or costly downtime [3,4]. Traditional fault diagnosis methods depend on expert knowledge and manual feature extraction [5–7]. Although machine learning-based methods reduce some of this dependency, they often still require carefully engineered features. In contrast, deep learning (DL)-based methods have demonstrated effectiveness by automatically extracting features from raw vibration signals for fault diagnosis, as discussed in survey papers [8–11] and references therein. For bearing fault diagnosis, various DL-based methods have been developed. For example, the authors of [12] proposed an bearing fault diagnosis method using vibration responses and convolutional neural network. Considering multiple data source, the authors of [13] developed an bearing fault diagnosis method across different working conditions using multiple source. Moreover, the authors of [14] presented an trackable multi-domain bearing fault diagnosis method for imbalanced data. However, in industrial applications, data are typically distributed across different organizations or facilities. These DL-based methods generally assume that large-scale

* Corresponding author.
E-mail address: yangtao@mail.neu.edu.cn (T. Yang).

https://doi.org/10.1016/j.ymssp.2026.113917
Received 24 July 2025; Received in revised form 14 December 2025; Accepted 19 January 2026
Available online 22 January 2026
0888-3270/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.



(a) Closed-set classification problem (b) Open-set classification problem

Fig. 1. Illustration of two classification problems: (a) closed-set classification, where trained decision boundaries effectively separate known categories, and (b) open-set classification, where unknown (gray) samples lead to misclassifications.

labeled data from different sources can be centralized on a single server, which is rarely feasible in industrial practice due to privacy constraints [15–17].

Federated learning (FL) has emerged as a promising solution to this challenge, as it allows multiple clients (organizations) to collaboratively train a shared global model without accessing local raw data [18,19]. Federated fault diagnosis (FFD) leverages FL to enable clients (e.g., factories, organizations) to train local fault diagnosis models on their private data by periodically sharing updates with a central server, which aggregates them to improve the global model [20,21]. This approach allows clients to benefit from the collective knowledge of distributed data sources without compromising data privacy [22,23]. Therefore, individual clients do not need to invest significant resources in acquiring labeled datasets, reducing the cost and effort. Recent studies have proposed several FFD methods, showing that FL can be applied effectively to enhance diagnostic performance while addressing privacy concerns. For example, the authors of [24] proposed a data privacy preserving FL method using prior distributions. Moreover, the authors of Xu et al. [25] and Russell and Wang [26] proposed FFD methods for rolling bearing based on cluster alignment and self-supervised learning, respectively.

Despite the progress in FFD, most of these works are limited to a closed-set scenario, where the categories of faults present in the testing data are the same as those in the training data [27,28]. However, this assumption may not always be satisfied. Unseen faults may occur during operation, a closed-set FL model therefore tends to map unknown faults to known classes with high confidence, as shown in Fig. 1, which leads to misclassifications and is unsafe. This mismatch between training and testing data highlights the need for recognizing unseen or unknown fault-a problem known as open-set fault diagnosis [29]. Several methods have been developed to solve this problem. For example, the authors of Chen et al. [30] proposed an open fault semantic subspace-based open-set fault diagnosis framework, and the authors of Lin and Zhao [31] realized open-set fault diagnosis of wind turbine gearbox based on Open-Set Kolmogorov-Arnold network. Moreover, the authors of Li et al. [32] proposed a fault diagnosis method based on adversarial training and the authors of Yu et al. [33] developed a bilateral weighted adversarial framework, respectively. However, most of these methods operate in a centralized setting, assuming access to all data at once.

In this paper, we propose a Federated Open-Set Fault Diagnosis (FOSFD) method. More specifically, during the training phase, inspired by MixUp [34], a technique that linearly interpolates pairs of samples and their labels to create augmented data and improve model generalization, we propose a MixUp-based feature augmentation strategy to generate synthetic features at local clients, thereby facilitating clearer decision boundaries between known and unknown samples. Moreover, we align the prototypes, which capture the underlying distributions of known categories, through the central server as well as model parameters. During the testing phase, by modeling the distances between the feature representations of samples and the selected prototypes, anomaly scores are calculated for test samples to determine whether the sample belongs to known categories. If it belongs to a known category, the well-trained classifier is used to output the classification result, otherwise it will be classified into an unknown category.

The main contributions of this paper are as follows:

- A federated open-set fault diagnosis method is proposed, which allows multiple clients to collaboratively detect unknown bearing faults without sharing raw vibration data. Comparing with the existing open-set fault diagnosis methods [30,33,35,36], our proposed method achieves the recognition in a distributed manner and protects the privacy.
- A MixUp-based feature augmentation strategy and a prototype alignment strategy are designed to regularize the feature distributions of known fault categories, where feature augmentation enlarges the margins and smooths the decision boundaries, and prototype alignment compacts the category prototypes, which can improve the rejection of unknown faults.
- The effectiveness of the proposed method is verified on two public bearing fault datasets [37,38], which shows that our proposed method outperforms exiting methods [28,33,39,40].

The rest of this paper is organized as follows. In Section 2, the problem formulation is presented. The proposed method and experimental results are presented in Section 3 and Section 4, respectively. Finally, Section 5 concludes the paper.

2. Problem formulation

2.1. Federated learning for fault diagnosis

We begin with formal definition of federated learning (FL) scenario for fault diagnosis, where a global central server G collaborates with C local clients. Each client $l \in \{1, \dots, C\}$ owns a private dataset that cannot be directly accessed by other clients or by the server. Let $\mathcal{X}$ denote the input space and $\mathcal{K}$ denote the set of known fault categories shared across all clients. For client l, there is a local closed-set training dataset $\mathcal{D}^l_{\text{close}} = \{(x^l_i, y^l_i)\}_{i=1}^{n_l} \in \mathcal{X} \times \mathcal{K}$, where (x^l_i, y^l_i) is the $i-$th sample with its label of client l, n_l is the number of samples of client l, and $\mathcal{X}$ represents the input space.

Let f_Θ be the global model with parameters Θ, and let $\ell(\cdot, \cdot)$ be the classification loss (e.g., cross-entropy). The goal of standard FL is to collaboratively minimize a weighted sum of local empirical risks:

$$\min_{\Theta} F(\Theta) := \sum_{l=1}^{C} \frac{n_l}{n} F_l(\Theta), \tag{1}$$

where

$$F_l(\Theta) = \mathbb{E}_{(x,y)\sim \mathcal{D}^l_{\text{close}}} \left[\ell\left(f_\Theta(x), y\right) \right] \tag{2}$$

is the local objective on client l, and $n = \sum_{i=1}^{C} n_i$ is the number of total samples. In practice, this objective is optimized by alternating between local training on each client and global aggregation on the server, such as the Federated Averaging (FedAvg) strategy, where only model parameters or gradients are communicated instead of raw data.

2.2. Federated open-set fault diagnosis

Then we extend conventional federated fault diagnosis to federated open-set fault diagnosis. Let $\mathcal{U}$ denote the set of unseen fault categories that are not present in the training phase, and $\mathcal{K} \cap \mathcal{U} = \varnothing$. During training, each client only observes labeled samples from the known categories $\mathcal{K}$.

During the testing phase, we consider an open-set scenario. Let $\mathcal{D}_{\text{test}} = \{(x_i, y_i)\}_{i=1}^{n_{\text{test}}} \in \mathcal{X} \times (\mathcal{K} \cup \mathcal{U})$ denote the testing dataset, where n_{test} is the number of test samples and each test label y_t may belong either to a known fault ($y_t \in \mathcal{K}$) or to an unknown fault ($y_t \in \mathcal{U}$).

In the federated open-set fault diagnosis problem considered in this paper, the objective is to collaboratively train a global model f_Θ across the C clients such that, for each test sample x_t, if $y_t \in \mathcal{K}$, the model correctly predicts its known fault category; if $y_t \in \mathcal{U}$, the model rejects the sample as an unknown fault rather than misclassifying it into any known category.

3. The proposed method

In this section, we introduce our proposed method, which operates in two phases: the training and the testing phases.

During the training phase, as shown in Fig. 2, each client operates on its private dataset with a shared model structure consisting of an encoder, a decoder, a discriminator, and a classifier. At each iteration, the local client first extracts feature representations of the samples using the encoder. Two primary losses are then calculated to update model parameters by back-propagation: the reconstruction loss derived from the encoder–decoder pair, and the empirical risk loss obtained from the classifier. Moreover, to obtain better decision boundaries, a feature augmentation strategy is employed. Through this strategy, augmented feature representations are generated and then the MixUp loss is obtained by the discriminator. In parallel, each client calculates local prototypes, representing the local central tendency of known categories based on the feature representations. These local prototypes are then aligned with global prototypes shared across all clients by the central server G. The prototype loss is derived from the discrepancy between the local features and its corresponding global prototypes. The training details and updating rules of local clients are given in Section 3.1 and the training details of the central server are provided in Section 3.2.

During the testing phase, as shown in Fig. 3, each local client evaluates test samples independently. For a given test sample, the encoder first generates its feature representation. Next, the distances between this feature representation and the global prototypes of known categories are calculated, resulting in $|\mathcal{K}|$ distance metrics, where $|\mathcal{K}|$ represents the number of known categories. These distances are then passed through the pre-fitted Weibull models, which outputs $|\mathcal{K}|$ anomaly scores, determining whether the sample belongs to known categories or not. The details of the testing phase are given in Section 3.3.

3.1. Training of local clients

During the training phase, local clients collaboratively work with the central server to train a global model. More specifically, both the clients and the server have the same model architecture to ensure consistency, as shown in Fig. 2. The model consists of four modules: an encoder $\text{En}(\cdot; \Theta_{\text{En}})$, a decoder $\text{De}(\cdot; \Theta_{\text{De}})$, a classifier $\text{Cls}(\cdot; \Theta_{\text{Cls}})$, and a discriminator $\text{Dis}(\cdot; \Theta_{\text{Dis}})$, where $\Theta_{\text{En}}, \Theta_{\text{De}}, \Theta_{\text{Cls}}$ and Θ_{Dis} represent the model parameters of each module, respectively.

At each iteration, clients locally updates their models through back-propagation, which involves several loss functions given respectively. First, the feature representations of samples are obtained through the encoder, and then the reconstruction loss L_{Re} (3)

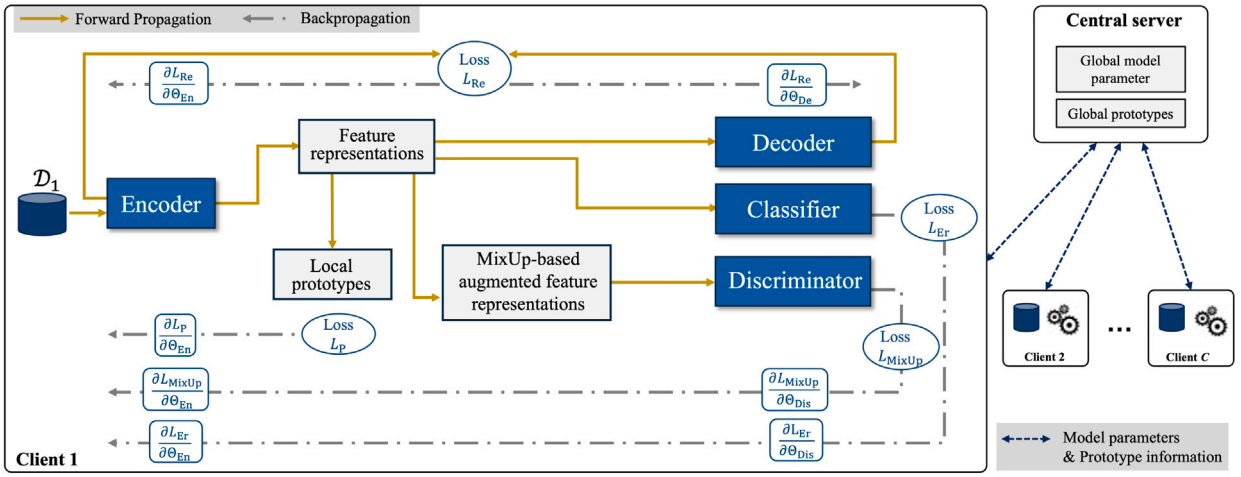


Fig. 2. Illustration of the proposed method's training phase.

and the empirical risk loss L_{Er} (6) are obtained through the decoder and the classifier, respectively. Then, MixUp-based augmented feature representations are generated, and the augmentation loss L_{MixUp} (9) is obtained through the discriminator. In addition, local prototypes are calculated based on the feature representations, and the prototype loss L_{P} (15) is obtained with the global prototypes. The model parameters are updated according to the above losses. The updated model parameters and local prototypes are then transmitted to the central server. Then the central server aggregates these updates to refine the global model.

3.1.1. Reconstruction loss

To enable the model to extract robust latent feature representations, an encoder–decoder pair is introduced. The encoder–decoder ensures that the feature representations preserves information essential for reconstructing input samples, thus improving the model's ability for downstream classification task.

The reconstruction loss L_{Re} of client l at iteration t, which measures the difference between the original input sample x_i and its reconstructed version $\bar{x}_i$, is defined as:

$$L_{\mathrm{Re}} = \frac{1}{n_l} \sum_{i=1}^{n_l} \|x_i - \bar{x}_i\|_2, \tag{3}$$

where n_l is the number of samples of client l, $\|\cdot\|_2$ denotes the Euclidean norm, and $\bar{x}_i$ is the reconstructed sample by decoding the feature representation z_i:

$$\bar{x}_i = \mathrm{De}(z_i; \Theta_{\mathrm{De}}(t)), \tag{4}$$

where $\Theta_{\mathrm{De}}(t)$ is the model parameters of Decoder De at iteration t, and the latent feature representation z_i of the sample (x_i, y_i) is computed by:

$$z_i = \mathrm{En}(x_i; \Theta_{\mathrm{En}}(t)), \tag{5}$$

where $\Theta_{\mathrm{En}}(t)$ is the model parameters of Encoder En at iteration t.

3.1.2. Empirical risk loss

The standard empirical risk minimization approach for classification is utilized. Given the latent representation z_i of sample x_i, the empirical risk loss L_{Er} for client l at iteration t is then defined as the negative log-likelihood of the true label y_i:

$$L_{\mathrm{Er}}(t) = \frac{1}{n_l} \sum_{i=1}^{n_l} \Big(-y_i \log\big(\mathrm{Cls}(z_i; \Theta_{\mathrm{Cls}}(t))\big) \Big), \tag{6}$$

where $\Theta_{\mathrm{Cls}}(t)$ is the model parameters of Classifier Cls at iteration t.

This loss helps the model minimize the classification error and improves generalization ability to unseen samples by adjusting model parameters of the encoder and the classifier.

3.1.3. Augmentation loss

To further improve the effectiveness of recognizing unseen samples, a feature-level MixUp augmentation strategy is applied. Given two different latent features representations z_i and z_j of samples (x_i, y_i) and (x_j, y_j), they are linearly mixed as follows:

$$\tilde{z}_i = \lambda z_i + (1 - \lambda) z_j, \tag{7}$$

$$\tilde{y}_i = \lambda y_i + (1-\lambda) y_j, \tag{8}$$

where $\lambda \in [0,1]$ is a mixing coefficient drawn from a Beta distribution: $\lambda \sim \mathrm{Beta}(1.2, 1.2)$.

The augmented feature $\tilde{z}_i$ and its corresponding label $\tilde{y}_i$ are then used to train the discriminator $\mathrm{Dis}(\cdot;\Theta_{\mathrm{Dis}})$. Therefore, the MixUp loss L_{MixUp} at iteration t is calculated by:

$$L_{\mathrm{MixUp}}(t) = \frac{1}{n_l}\sum_{i=1}^{n_l}\Big(-\tilde{y}_i \log\big(\mathrm{Dis}(\tilde{z}_i;\Theta_{\mathrm{Dis}}(t))\big) + (1-\tilde{y}_i)\log\big(1-\mathrm{Dis}(\tilde{z}_i;\Theta_{\mathrm{Dis}}(t))\big)\Big), \tag{9}$$

where $\mathrm{Dis}(\cdot;\Theta_{\mathrm{Dis}}(t))$ is the Discriminator of client l with parameters $\Theta_{\mathrm{Dis}}(t)$.

This augmentation loss encourages the discriminator to recognize mixed feature representations while enhancing the encoder's ability to generate latent feature representations that are resilient to interpolation noise, thus improving decision boundaries of the model.

3.1.4. Prototype alignment loss

To further improve the decision boundaries of the global model, a prototype alignment strategy is introduced. Category-specific prototype representations for each client are maintained. More specifically, the prototype $p_{l,c}$ for category c on client l at iteration t is defined as:

$$p_{l,c}(t) = \frac{1}{\bar{n}_{l,c}}\sum_{i=1}^{n_l}\mathbb{1}(\bar{y}_i = y_i)\mathbb{1}(y_i = c)\mathrm{En}(x_i;\Theta_{\mathrm{En}}(t)), \tag{10}$$

where $\bar{n}_{l,c}$ is the number of samples which are corrected classified into category c by the classifier in client l, $\mathbb{1}(\cdot)$ is the indicator function, and $\bar{y}_i$ is the predicted label for sample x_i:

$$\bar{y}_i = \mathrm{Cls}(z_i;\Theta_{\mathrm{Cls}}(t)). \tag{11}$$

The local prototypes $\{p_{l,c}(t)\}_{c\in\mathcal{K}}$ are updated iteratively, and the prototype alignment loss L_{P}, which will be detailed in (15), is introduced to reduce the divergence between client-specific prototypes and global prototypes.

3.1.5. Updating rules at local clients

The following update rules is proposed for the parameters of client l at iteration t to balance various loss functions:

$$\Theta^l_{\mathrm{En}}(t+1) = \Theta^l_{\mathrm{En}}(t) - \epsilon(t)\Big(\alpha\frac{\partial L_{\mathrm{Re}}(t)}{\partial\Theta^l_{\mathrm{En}}(t)} + \beta\frac{\partial L_{\mathrm{Er}}(t)}{\partial\Theta^l_{\mathrm{En}}(t)} + \gamma\frac{\partial L_{\mathrm{MixUp}}(t)}{\partial\Theta^l_{\mathrm{En}}(t)} + \frac{\partial L_{\mathrm{P}}(t)}{\partial\Theta^l_{\mathrm{En}}(t)}\Big), \tag{12a}$$

$$\Theta^l_{\mathrm{De}}(t+1) = \Theta^l_{\mathrm{De}}(t) - \epsilon(t)\Big(\alpha\frac{\partial L_{\mathrm{Re}}(t)}{\partial\Theta^l_{\mathrm{De}}(t)}\Big), \tag{12b}$$

$$\Theta^l_{\mathrm{Cls}}(t+1) = \Theta^l_{\mathrm{Cls}}(t) - \epsilon(t)\Big(\beta\frac{\partial L_{\mathrm{Er}}(t)}{\partial\Theta^l_{\mathrm{Cls}}(t)}\Big), \tag{12c}$$

$$\Theta^l_{\mathrm{Dis}}(t+1) = \Theta^l_{\mathrm{Dis}}(t) - \epsilon(t)\Big(\gamma\frac{\partial L_{\mathrm{MixUp}}(t)}{\partial\Theta^l_{\mathrm{Dis}}(t)}\Big), \tag{12d}$$

where iteration $t = 0, 1, 2, \ldots$, $\epsilon(t)$ is the learning rate at iteration t, where α, β, and γ are non-negative hyperparameters that balance the contributions of the reconstruction loss, empirical risk loss, and MixUp loss, respectively. Moreover, the model parameters $\Theta^l_{\mathrm{En}}(0), \Theta^l_{\mathrm{De}}(0), \Theta^l_{\mathrm{Cls}}(0)$ and $\Theta^l_{\mathrm{Dis}}(0)$ can be chosen arbitrarily, and L_{P} is the prototype alignment loss, which will be detailed in (15).

After local updates, the updated model parameters are uploaded to the central server for aggregation.

3.2. Global aggregation at the central server

After each round of local training, the central server receives updates from each client and conducts two major tasks: (i) integration of model parameters to form the global model parameters, and (ii) integration of prototypes to maintain consistency across clients.

3.2.1. Model parameter aggregation

Clients perform local training based on their local data and send their updated model parameters to the central server. The central server then aggregates these parameters using a weighted averaging scheme, such as FedAvg [18], where the weight of each client's contribution is proportional to the size of its local dataset. Let $\Theta^l(t) = \{\Theta^l_{\mathrm{En}}(t), \Theta^l_{\mathrm{De}}(t), \Theta^l_{\mathrm{Cls}}(t), \Theta^l_{\mathrm{Dis}}(t)\}$ represent the parameters of client l at iteration t. The global model parameters $\Theta^g(t)$ are updated as:

$$\Theta^g(t) = \frac{\sum_{l=1}^{C} n_l \Theta^l(t)}{\sum_{l=1}^{C} n_l}. \tag{13}$$

Algorithm 1 Training of FOSFD

Input: The number of clients C, the set of known categories $\mathcal{K}$, the training dataset D_{close}, the number of iteration E.
Initialization
  Client Operations
    for each client $l = 1, \cdots, C$ **do**
      Initialize $\Theta^l(0)$;
      Calculate $\{p_{l,c}(0)\}_{c\in\mathcal{K}}$ by (10);
      Sent $\Theta^l(0)$ and $\{p_{l,c}(0)\}_{c\in\mathcal{K}}$ to the server;
    end for
  Server Operations
    Receive $\{\Theta^l(0)\}_{l=1}^{C}$ and $\{\{p_{l,c}(0)\}_{c\in\mathcal{K}}\}_{l=1}^{C}$;
    Aggregate model parameters to get $\Theta^g(0)$ by (13);
    Aggregate prototypes to get $\{\bar{P}_c(0)\}_{c\in\mathcal{K}}$ by (14);
    Send $\Theta^g(0)$ and $\{\bar{P}_c(0)\}_{c\in\mathcal{K}}$ to each client;
Training Process
  for iteration $t = 1, \cdots, E$ **do**
    Client Operations
      for each client $l = 1, \cdots, C$ **do**
        Receive $\Theta^g(t-1)$ and $\{\bar{P}_c(t-1)\}_{c\in\mathcal{K}}$;
        $\Theta^l(t) \leftarrow \Theta^g(t-1)$;
        Update $\Theta^l(t)$ by (12);
        Calculate $\{p_{l,c}(t)\}_{c\in\mathcal{K}}$ by (10);
        Sent $\Theta^l(t)$ and $\{p_{l,c}(t)\}_{c\in\mathcal{K}}$ to the server;
      end for
    Server Operations
      Receive $\{\Theta^l(t)\}_{l=1}^{C}$ and $\{\{p_{l,c}(t)\}_{c\in\mathcal{K}}\}_{l=1}^{C}$;
      Aggregate model parameters to get $\Theta^g(t)$ by (13);
      Aggregate prototypes to get $\{\bar{P}_c(t)\}_{c\in\mathcal{K}}$ by (14);
      Send $\Theta^g(t)$ and $\{\bar{P}_c(t)\}_{c\in\mathcal{K}}$ to each client;
  end for
Output: $\Theta^g(E)$, $\{\bar{P}_c(E)\}_{c\in\mathcal{K}}$.

3.2.2. Prototype alignment for global representation consistency

The central server is responsible for aggregating local prototypes to facilitate alignment across clients. The global prototype for category c at iteration t, denoted as $\bar{P}_c(t)$, is calculated using a weighted average across clients as follows:

$$\bar{P}_c(t) = \frac{\sum_{l=1}^{C} \bar{n}_{l,c} p_{l,c}(t)}{\sum_{l=1}^{C} \bar{n}_{l,c}}, \tag{14}$$

where $\bar{n}_{l,c}$ denotes the number of correctly classified samples belonging to category c in client l, and $p_{l,c}(t)$ is the prototype of category c on client l at iteration t.

The new calculated global prototypes $\{\bar{P}_c(t)\}_{c\in\mathcal{K}}$ will be sent to local clients, therefore, the prototype alignment loss L_{P} for client l at iteration t is calculated by:

$$L_{\text{P}}(t) = \sum_{c\in\mathcal{K}} \left(\frac{1}{\sum_{i=1}^{n_l} \mathbb{1}(y_i = c)} \sum_{i=1}^{n_l} \mathbb{1}(y_i = c)(\|z_i - \bar{P}_c(t)\|_2) \right). \tag{15}$$

By aggregating prototypes in this manner, local feature representations would be closer to the global prototypes, thereby mitigating the effects of local biases and improving cross-client model consistency.

The above training phase is summarized in pseudo-code as Algorithm 1.

3.3. Extreme value theory-based testing phase

The inference process of the FOSFD method on the local client l contains two steps: (i) building extreme-value-theory (EVT) [41]-based Weibull distribution models and (ii) determining the output based on anomaly scores calculated by Weibull distribution models, as illustrated in Fig. 3. First, the test sample is fed into the encoder to get the feature expression. The distance between the feature representation and each prototype is calculated, and C distances are obtained and brought into fitted Weibull models to get C scores. Then, the scores are compared with the threshold. If any of the scores are greater than the threshold, the sample is classified as an unknown category, otherwise the feature representation is fed into the classifier to get the classification result.

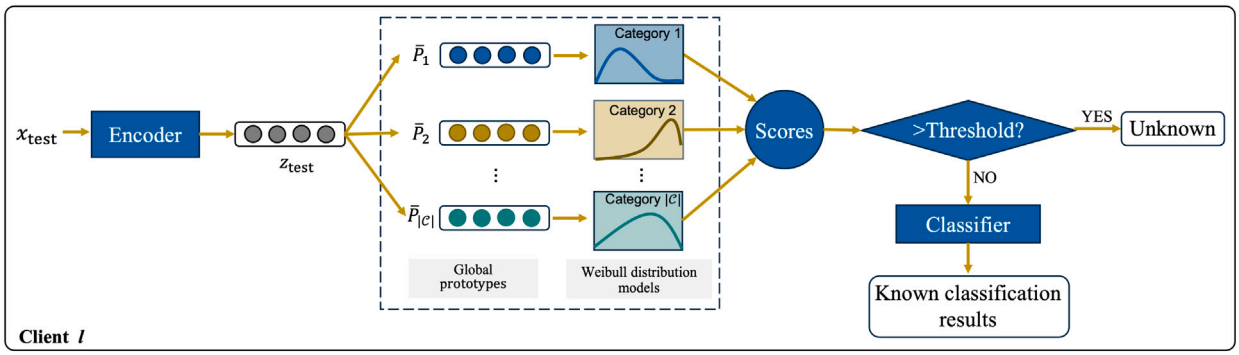


Fig. 3. Illustration of the proposed method's testing phase on local client l.

3.3.1. EVT-based Weibull distribution models

After E iterations of training, the final global protutypes $\{\bar{P}_c(E)\}_{c\in\mathcal{K}}$ for each known category $c \in \mathcal{K}$ is obtained. The Weibull distribution models are built by the distances between each training sample's feature representations and its corresponding global prototype.

First, for each training sample x_i belonging to category $c \in \mathcal{K}$, the distance $d_{i,c}$ between the sample's feature representation z_i and the global prototype $\bar{P}_c(E)$ is calculated using the Euclidean norm:

$$d_{i,c} = \|z_i - \bar{P}_c(E)\|_2. \tag{16}$$

Next, the Weibull distribution parameters $\rho_c = (\tau_c, \kappa_c, \lambda_c)$ are fit to the set of distances $d_{l,c} = \{d_{i,c}\}_{i=1}^{n_{l,c}}$ for each category c on client l, using a tail size μ, where τ_c, λ_c, and κ_c are the Weibull distribution parameters and $n_{l,c}$ is the number of samples belonging to categories c on local client l. The tail size μ controls the portion of the distance distribution used to fit the Weibull distribution. This process results in per-category Weibull distribution models, i.e., $|\mathcal{K}|$ Weibull distribution models, capturing the distribution of feature representation distances for each category.

3.3.2. Output based on anomaly scores

For each test sample $x_t \in \mathcal{D}_{\text{test}}$, get the feature representations z_t, and calculate the distance $d_{t,c}$ between z_t and the global prototype $\bar{P}_c(E)$ for each category $c \in \mathcal{K}$. Thus, the anomaly score $\omega(d_{t,c})$ on category c, which reflects how likely the test sample belongs to category c, is calculated using Weibull cumulative distribution function:

$$\omega(d_{t,c}) = 1 - \exp\left(-\left(\frac{d_{t,c} - \tau_c}{\lambda_c}\right)\right)^{\kappa_c}. \tag{17}$$

If for any category c, the anomaly score $\omega(d_{t,c})$ exceeds a threshold Ω, the test sample is rejected as an unknown category, meaning it does not belong to any of the known categories. Otherwise, the feature representations z_t of test sample x_t is fed into the classifier to get the predicted result:

$$\bar{y}_t = \text{Cls}(z_t; \Theta^g_{\text{Cls}}(E)). \tag{18}$$

The testing phase of the proposed method is summarized in pseudo-code as Algorithm 2.

4. Experimental results

In order to verify the effectiveness of the proposed method, experiments are carried out on Case Western Reserve University (CWRU) [37] and Paderborn University (PU) [38] bearing datasets, respectively.

4.1. Datasets

4.1.1. CWRU dataset

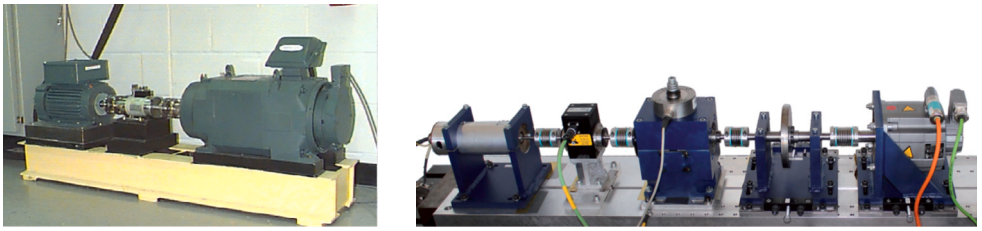
CWRU dataset [37] has ten categories, including one normal (NO) and nine faulty bearings data, where faults include inner faults (IF), outer faults (OF) and rolling ball faults (BF). Moreover, each fault is divided into three categories according to different fault diameters (7, 14, and 21 mils). The vibration signals were collected from the driven end of motor (see Fig. 4(a)) at a sampling frequency of 12,000 Hz.

4.1.2. PU dataset

PU dataset [38] contains vibration data collected on the bearing test rig (see Fig. 4(b)) from healthy, artificial damage and real damage bearings at a sampling frequency of 64,000 Hz. Moreover, PU dataset is downsampled to the same frequency as the CWRU dataset. Real damage bearing data are used, including four categories (NO, IF, OF, and mixed faults by IF and OF).

Algorithm 2 Testing of FOSFD on Local Client l

Input: Feature representations of local training dataset $\{z_i\}_{i=1}^{n_l}$, tail size μ, threshold Ω, test set $\mathcal{D}_{\text{test}} = \{x_j\}_{j=1}^{n_t}$, and global prototypes $\{\bar{P}_c(E)\}_{c\in\mathcal{K}}$.
Build Weibull distribution models:
  for each category $c \in \mathcal{K}$ **do**
    For each training sample on client l, compute distance to $\bar{P}_c(E)$ by (16) to get the distance set $d_{l,c}$;
    Built Weibull distribution model for category c;
  end for
Get prediction results:
Get feature representation z_t for each sample $x_t \in \mathcal{D}_{\text{test}}$;
  for each category $c \in \mathcal{K}$ **do**
    Compute distance by (16);
    Calculate anomaly score $\omega(d_{t,c})$ by (17);
  end for
Reject if $\omega(d_{t,c}) > \Omega$ for any category c.
Else output: Compute $\bar{y}_j$ by (18).



(a) (b)

Fig. 4. Experimental test rigs of (a) CWRU bearing dataset [37] and (b) PU bearing dataset [38].

4.2. Experimental setup

For both CWRU and PU datasets, we design three fault detection tasks. For the CWRU dataset with 10 categories, (i) normal data and three random fault categories, (ii) normal and four random fault categories, and (iii) normal and five random fault categories are selected as known categories in the experiments, respectively, and the rest data are as unseen categories. For the PU dataset with four categories, (i) normal and inner ring fault, (ii) normal and outer ring fault, and (iii) normal, inner and outer ring fault are as known categories in the experiments, respectively.

Moreover, due to the testing phase is conducted on each client, the result of one task is the mean of all clients' results. To ensure a reliable evaluation of stability, each task is repeated ten times and the average result is reported.

4.3. Comparison methods and evaluation metrics

The effectiveness of the proposed method is verified by comparing with four exiting methods, including SOSFD [33], FedPD [40], OpenMax [28] and PEOSER [39]. Since SOSFD, OpenMax, PROSER are not suitable for FL assumption, FedAvg [18] is introduced to combine with these methods for a fair comparison.

In open-set fault diagnosis, the task is not only to classify samples into known categories but also to detect unknown categories. Therefore, we adopt (i) Accuracy (Acc), (ii) Area Under the Curve (AUC) [42], and (iii) Open Set Classification Rate (OSCR) [43] as the evaluation metrics for both closed-set classification and open-set recognition.

4.4. Implementation details

PyTorch framework is utilized and all experiments are performed on a GeForce RTX 3090 graphics card. Moreover, the vibration data are divided into samples with the size of time window setting to 1024, and each sample is normalized to $0-1$. Down sampling is conducted on PU dataset to ensure the same sampling frequency of 12,000 Hz. For each dataset, we construct the federated setting by creating $K = 10$ clients. The training data are randomly divided into ten disjoint subsets, and each client receives one subset as its private local data.

The backbone of the network is detailed in Table 1. Batch size for experimental is set to 64 for both CWRU and PU dataset. Adam [44] is used to optimize networks. The learning rate is set to 10^{-3}. Hyper-parameters α, β and γ in (12) are set to 0.1, 0.1, and 0.1, respectively. For a fair comparison, we use the same model parameter settings in all experiments.

Table 1
Architecture details of the model.

Modules	Layer	Parameters
Encoder	Conv1D	Kernels/size/stride: 15/16/1
	BN	–
	ReLU	–
	Conv1D	Kernels/size/stride: 3/32/1
	BN	–
	ReLU	–
	MaxPool	Kernels/stride: 2/2
	Conv1D	Kernels/size/stride: 3/128/1
	BN	–
	ReLU	–
	Adaptive MaxPool	4
	FC	input/output: 512/256
	ReLU	–
	FC	input/output: 256/256
	ReLU	–
Decoder	FC	input/output:256/256
	BN	–
	LeakyReLU	–
	FC	input/output:256/512
	BN	–
	LeakyReLU	–
	FC	input/output:512/1024
	Sigmoid	–
Discriminator	FC	input/output: 256/256
	ReLU	–
	Dropout	0.5
	FC	input/output: 256/256
	ReLU	–
	Dropout	0.5
	FC	input/output: 256/1
	Sigmoid	–
Classifier	FC	input/output: 256/$\|\mathcal{K}\|$

Table 2
Accuracy, AUC, and OSCR results of open set tasks on CWRU dataset.

Method	$\|\mathcal{K}\| = 4$			$\|\mathcal{K}\| = 5$			$\|\mathcal{K}\| = 6$			Avg.		
	Closed-Acc	Open-AUC	OSCR	Closed-Acc	Open-AUC	OSCR	Closed-Acc	Open-AUC	OSCR	Closed-Acc	Open-AUC	OSCR
SOSFD [33]	0.995	0.681	0.692	0.997	0.649	0.641	1.000	0.597	0.588	0.997	0.642	0.640
FedPD [40]	1.000	0.554	0.436	0.997	0.684	0.669	1.000	0.671	0.656	0.999	0.636	0.587
OpenMax [28]	0.997	0.789	0.808	0.998	0.849	0.830	0.995	0.787	0.783	0.997	0.809	0.807
PROSER [39]	1.000	0.658	0.672	0.995	0.689	0.742	1.000	0.759	0.640	0.998	0.702	0.685
FOSFD (Ours)	0.997	**0.857**	**0.868**	0.996	**0.875**	**0.873**	0.998	**0.861**	**0.853**	0.997	**0.864**	**0.865**

Table 3
Accuracy, AUC, and OSCR results of open set tasks on PU dataset.

Method	$\mathcal{K} = \{NO, IF\}$			$\mathcal{K} = \{NO, OF\}$			$\mathcal{K} = \{NO, IF, OF\}$			Avg.		
	Closed-Acc	Open-AUC	OSCR	Closed-Acc	Open-AUC	OSCR	Closed-Acc	Open-AUC	OSCR	Closed-Acc	Open-AUC	OSCR
SOSFD [33]	0.996	0.592	0.566	1.000	0.705	0.715	0.994	0.602	0.624	0.997	0.633	0.635
FedPD [40]	0.995	0.541	0.565	1.000	**0.794**	0.781	0.996	0.606	0.636	0.997	0.647	0.661
OpenMax [28]	1.000	0.568	0.537	0.996	0.762	0.728	1.000	0.691	0.732	0.999	0.674	0.666
PROSER [39]	0.996	0.543	0.584	1.000	0.551	**0.827**	0.994	0.612	0.688	0.997	0.569	0.700
FOSFD (Ours)	0.995	**0.621**	**0.604**	1.000	0.774	0.757	0.994	**0.725**	**0.786**	0.996	**0.707**	**0.716**

4.5. Result analysis

4.5.1. Results on CWRU dataset

The results of various methods on CWRU dataset across different tasks are summarized in Table 2. For each task, we evaluate the closed-set Accuracy (Close-Acc), open-set AUC (Open-AUC) and OSCR metrics. In terms of Close-Acc, all methods achieve near-perfect performance, indicating that closed-set classification on known fault categories is relatively easy for this dataset. The main differences appear in Open-AUC and OSCR, which reflect the ability to detect and reject unknown faults. SOSFD attains an average Open-AUC of only 0.642 and an OSCR of 0.640, while FedPD yields a comparable average Open-AUC of 0.636 but the lowest average OSCR of 0.587, suggesting that its decision boundaries tend to misclassify unknown samples even when closed-set accuracy is high. OpenMax reaches an average Open-AUC of 0.809 and OSCR of 0.807, while PROSER achieves 0.702 and 0.685, respectively. Our

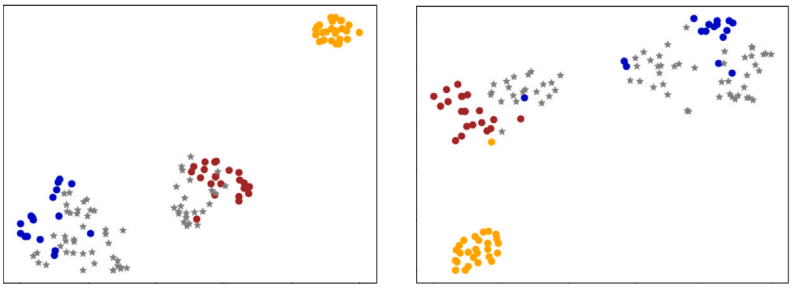


Fig. 5. t-SNE visualization of the feature spaces (a) before and (b) after augmentation on task $\mathcal{K}$ = {NO, IF, OF} of PU dataset.

proposed FOSFD delivers the best open-set performance across all tasks. It achieves the highest average Open-AUC of 0.864 and the highest average OSCR of 0.865. These results demonstrate that FOSFD not only preserves excellent closed-set recognition, but also substantially enhances the joint ability to correctly classify known faults and reject unknown ones.

4.5.2. Results on PU dataset

The results on PU dataset with varying numbers of known categories ($\mathcal{K}$ = {NO, IF}, {NO, OF}, {NO, IF, OF}) are presented in Table 3. Similar to CWRU dataset, all methods remains consistently high across all tasks in closed-set fault diagnosis, which indicates that the classification of known bearing conditions on PU dataset is not a major challenge. SOSFD and FedPD obtain relatively low average Open-AUC values of 0.633 and 0.647, and average OSCR values of 0.635 and 0.661, respectively. This shows that these two methods have difficulty rejecting unknown faults on PU dataset, especially in the more complex setting $\mathcal{K}$ = NO, IF, OF, where their Open-AUC and OSCR remain around 0.60–0.64. OpenMax improves the average Open-AUC to 0.674 and the average OSCR to 0.666, and PROSER achieves the highest OSCR in the $\mathcal{K}$ = NO, OF task (0.827), but its average Open-AUC is the lowest among all methods (0.569), indicating that its advantage is not consistent across different choices of known categories. Our FOSFD method achieves the best overall open-set performance. It attains the highest average Open-AUC of 0.707 and the highest average OSCR of 0.716 while maintaining a competitive Close-Acc of 0.996. In the most challenging configuration $\mathcal{K}$ = NO, IF, OF, FOSFD reaches an Open-AUC of 0.725 and an OSCR of 0.786, clearly outperforming all competing methods on both metrics. Compared with the CWRU results, the Open-AUC and OSCR values on PU are generally lower for all methods, which reflects the greater complexity and variability of the PU dataset. Even under these more demanding conditions, FOSFD provides the most balanced performance by simultaneously preserving high closed-set accuracy and improving the detection and rejection of unknown categories.

4.6. Ablation study

Ablation study is conducted to investigate the impacts of components in our proposed method. Moreover, the choice of hyper-parameters is also analyzed.

4.6.1. Effects of each component

We consider four key variations, the removal of (i) MixUp and discriminator (w/o MixUp), (ii) Decoder (w/o Decoder), and (iii) feature-based EVT inference (w/o EVT inference), in which softmax is the alternative to the feature-based EVT inference. The results of removing these components on CWRU and PU datasets are presented in Tables 4 and 5, respectively. As shown in Table 4, our proposed FOSFD achieves the highest Open-AUC scores with an average Open-AUC of 0.8641, while removing each component consistently degrades performance. When both the Decoder and MixUp modules are removed, the Open-AUC decreases significantly. Similarly, the removal of only MixUp results in an Open-AUC drop, with average Open-AUC at 0.8521. The exclusion of Decoder also yields performance decreases. Moreover, removing EVT inference leads to an average Open-AUC of 0.8432. The ablation results on the PU dataset (Table 5) exhibit similar trends. Our proposed FOSFD model achieves the highest average Open-AUC of 0.707, outperforming all ablated variants. These results demonstrate that all components of FOSFD collectively enhance the detection performance.

Moreover, Fig. 5 presents the t-SNE [45] visualization of the feature spaces of the PU test samples for task $\mathcal{K}$ = NO, IF, OF before and after the MixUp-based augmentation. Colored points correspond to known categories, while gray stars represent the unknown category. After augmentation, the known classes form more compact and well-separated clusters, and the unknown samples lie further away from the dense cores of the known clusters. This indicates that the MixUp-based feature augmentation leads to a more discriminative feature space, thereby facilitating the correct rejection of unknown samples.

Table 4
Ablation results on CWRU dataset.

Method	$\|\mathcal{K}\| = 4$		$\|\mathcal{K}\| = 5$		$\|\mathcal{K}\| = 6$		Avg.	
	Closed-Acc	Open-AUC	Closed-Acc	Open-AUC	Closed-Acc	Open-AUC	Closed-Acc	Open-AUC
w/o Decoder & MixUp	0.996	0.680	0.998	0.651	1.000	0.592	0.998	0.641
w/o MixUp	1.000	0.856	0.998	0.866	1.000	0.834	0.999	0.852
w/o Decoder	0.997	0.842	0.996	0.868	0.998	0.829	0.997	0.846
w/o EVT inference	0.997	0.846	0.996	0.859	0.998	0.825	0.997	0.843
FOSFD (Ours)	0.997	**0.857**	0.996	**0.875**	0.998	**0.861**	0.997	**0.864**

Table 5
Ablation Results on PU Dataset.

Method	$\mathcal{K}$ = {NO, IF}		$\mathcal{K}$ = {NO, OF}		$\mathcal{K}$ = {NO, IF, OF}		Avg.	
	Closed-Acc	Open-AUC	Closed-Acc	Open-AUC	Closed-Acc	Open-AUC	Closed-Acc	Open-AUC
w/o Decoder & MixUp	0.996	0.561	0.998	0.727	0.994	0.717	0.996	0.668
w/o MixUp	0.996	0.595	0.999	0.721	0.994	0.704	0.996	0.673
w/o Decoder	0.999	0.584	1.000	0.734	1.000	0.698	0.999	0.672
w/o EVT inference	0.996	0.582	1.000	0.658	0.995	0.619	0.997	0.620
FOSFD (Ours)	0.995	**0.621**	1.000	**0.774**	0.994	**0.725**	0.996	**0.707**

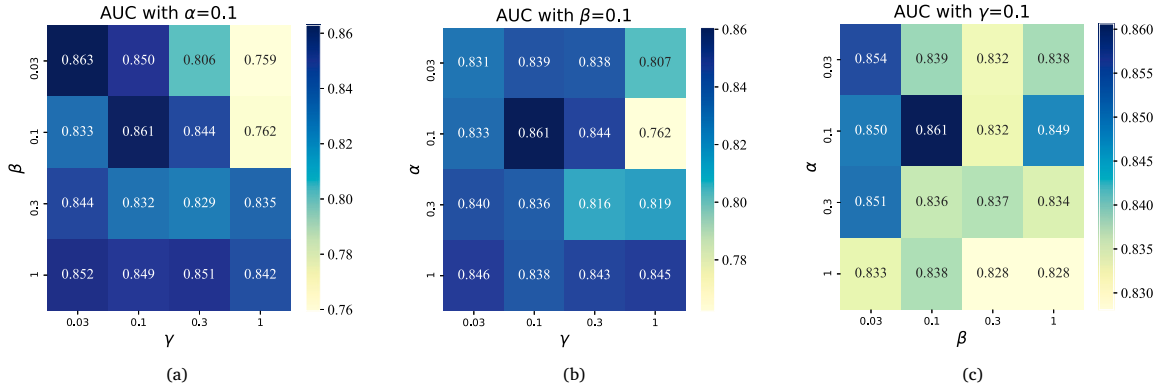


Fig. 6. Hyperparameter heatmaps on CWRU task $|\mathcal{K}| = 6$ with fixed hyperparameters (a) $\alpha = 0.1$, (b) $\beta = 0.1$, and (c) $\gamma = 0.1$, respectively.

4.6.2. Choice of hyperparameters

The impact of varying the hyperparameters α, β and γ on model performance is analyzed. Fig. 6 visualizes the effect of the three loss weights α, β, and γ on model performance by fixing one coefficient at 0.1 and sweeping the other two. The heatmaps display varying shades of blue to indicate AUC levels, with darker shades reflecting higher AUC. In the proposed framework, α scales the reconstruction term L_{Re}, β scales the classification term L_{Er}, and γ scales the MixUp regularization term L_{MixUp}. When α is fixed at 0.1 (Fig. 6(a)), varying β and γ within [0.03, 1] leads to modest AUC variations for most combinations, with the best region around $\beta \approx 0.1$ and $\gamma \approx 0.1$, and a noticeable drop only at the extreme $\gamma = 1$. The reasons for this phenomenon is the losses L_{Re}, L_{Er}, and L_{MixUp} have comparable scales during training, and the prototype term L_{P} acts as a strong anchor that stabilizes the feature space across clients. Only extreme settings — especially a large γ — over-smooth the decision boundary due to excessive MixUp regularization and thus reduce AUC. Similar plateaus are observed when fixing $\beta = 0.1$ (Fig. 6(b)) or $\gamma = 0.1$ (Fig. 6(c)): balanced weights around 0.1 yield the best or near-best performance, whereas extreme values—particularly a large γ—slightly degrade AUC. Consistent with these observations and the ablation results, we adopt $\alpha! = !\beta! = !\gamma! = !0.1$ as the default setting, which provides strong open-set performance while maintaining competitive closed-set accuracy.

5. Conclusion

In this paper, a fault diagnosis method called FOSFD is proposed to deal with the federated open-set fault diagnosis problem. Augmented feature representations are generated at each local client through MixUp-based feature augmentation strategy. The selected prototypes are aligned between local clients by the central server. These strategies enhance decision boundaries of the global model, so that the well-trained global model can effectively classify samples belonging to known categories correctly and also identify unknown samples. The effectiveness of our proposed method has been verified on two bearing datasets.

The proposed method assumes that the distribution of training and testing dataset are the same. However, if there exists the domain shift, the effectiveness may decrease. In the future, we will extend the proposed framework to address open-set fault diagnosis problem under domain shift between training and testing conditions. Moreover, how to use newly acquired unknown samples to improve the model is also a direction worth exploring.

CRediT authorship contribution statement

Danya Xu: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Conceptualization. **Mingwei Jia:** Writing – review & editing. **Tao Chen:** Writing – review & editing, Supervision. **Yi Liu:** Writing – review & editing. **Dongyue Chen:** Writing – review & editing. **Tianyou Chai:** Writing – review & editing. **Tao Yang:** Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported in part by the National Natural Science Foundation of China under Grants 62133003.

Data availability

The authors do not have permission to share data.

References

[1] R. Liu, B. Yang, E. Zio, X. Chen, Artificial intelligence for fault diagnosis of rotating machinery: A review, Mech. Syst. Signal Process. 108 (2018) 33–47.
[2] Y. Lei, B. Yang, X. Jiang, F. Jia, N. Li, A.K. Nandi, Applications of machine learning to machine fault diagnosis: A review and roadmap, Mech. Syst. Signal Process. 138 (2020) 106587.
[3] S.A. Aburakhia, R. Myers, A. Shami, A hybrid method for condition monitoring and fault diagnosis of rolling bearings with low system delay, IEEE Trans. Instrum. Meas. 71 (2022) 1–13.
[4] L. Feng, C. Zhao, Fault description based attribute transfer for zero-sample industrial fault diagnosis, IEEE Trans. Ind. Inf. 17 (2021) 1852–1862.
[5] R. Sharifi, R. Langari, Nonlinear sensor fault diagnosis using mixture of probabilistic pca models, Mech. Syst. Signal Process. 85 (2017) 638–650.
[6] H. Badihi, Y. Zhang, B. Jiang, P. Pillay, S. Rakheja, A comprehensive review on signal-based and model-based condition monitoring of wind turbines: Fault diagnosis and lifetime prognosis, Proc. IEEE 110 (2022) 754–806.
[7] W. Zhang, N. Jiang, S. Yang, X. Li, Federated transfer learning for remaining useful life prediction in prognostics with data privacy, Meas. Sci. Technol. 36 (2025) 076107.
[8] Z. Gao, C. Cecati, S.X. Ding, A survey of fault diagnosis and fault-tolerant techniques—part I: Fault diagnosis with model-based and signal-based approaches, IEEE Trans. Ind. Electr. 62 (2015) 3757–3767.
[9] Y. Chi, Y. Dong, Z.J. Wang, F.R. Yu, V.C.M. Leung, Knowledge-based fault diagnosis in industrial internet of things: A survey, IEEE Int. Things J 9 (2022) 12886–12900.
[10] P. Zhou, S. Chen, Q. He, D. Wang, Z. Peng, Rotating machinery fault-induced vibration signal modulation effects: A review with mechanisms, extraction methods and applications for diagnosis, Mech. Syst. Signal Process. 200 (2023) 110489.
[11] P. Gangsar, R. Tiwari, Signal based condition monitoring techniques for fault detection and diagnosis of induction motors: A state-of-the-art review, Mech. Syst. Signal Process. 144 (2020) 106908.
[12] S. Patil, V.G. Salunkhe, P.S. Jadhav, S. Khot, S.R. Desavale, R. Desavale, A novel bearing faults diagnosis of rotor-bearing systems based on vibration responses and convolutional neural network, Mech. Syst. Signal Process. 236 (2025) 113055.
[13] D. Xu, Y. Li, Y. Song, L. Jia, Y. Liu, IFDS: An intelligent fault diagnosis system with multisource unsupervised domain adaptation for different working conditions, IEEE Trans. Instrum. Meas. 70 (2021) 1–10.
[14] X. Wang, H. Jiang, M. Mu, Y. Dong, A trackable multi-domain collaborative generative adversarial network for rotating machinery fault diagnosis, Mech. Syst. Signal Process. 224 (2025) 111950.
[15] Y. Yuan, J. Liu, D. Jin, Z. Yue, T. Yang, R. Chen, et al., DeceFL: A principled fully decentralized federated learning framework, Natl. Sci. Open 2 (2023) 20220043.
[16] L. Fang, J. Shi, H. Qu, M. Safran, J. Tan, J. Wan, Contrastive prototype guided federated learning for rotating machinery fault diagnosis under spatio-temporal domain shift, Mech. Syst. Signal Process. 232 (2025) 112707.
[17] Q. Qian, B. Zhang, C. Li, Y. Mao, Y. Qin, Federated transfer learning for machinery fault diagnosis: A comprehensive review of technique and application, Mech. Syst. Signal Process. 223 (2025) 111837.
[18] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, 2017, pp. 1273–1282.
[19] W. Zhang, Z. Wang, X. Li, Blockchain-based decentralized federated transfer learning methodology for collaborative machinery fault diagnosis, Reliab. Eng. Syst. Saf. 229 (2023) 108885.
[20] Z. Yao, C. Zhao, FedTMI: Knowledge aided federated transfer learning for industrial missing data imputation, J. Process Control 117 (2022) 206–215.
[21] D. Xu, Y. Liu, G. Wen, Y. Jin, T. Chai, T. Yang, DeFedTL: A decentralized federated transfer learning method for fault diagnosis, IEEE Trans. Ind. Informatics 21 (2024b) 1704–1713.
[22] S. Abdulrahman, H. Tout, H. Ould-Slimane, A. Mourad, C. Talhi, M. Guizani, A survey on federated learning: The journey from centralized to distributed on-site learning and beyond, IEEE Internet Things J. 8 (2021) 5476–5497.
[23] Z. Xu, W.X. Zheng, Y. Wang, D. Wang, Z. Wu, Federated learning-based distributed model predictive control, J. Process Control 152 (2025) 103472.

[24] W. Zhang, X. Li, Data privacy preserving federated transfer learning in machinery fault diagnostics using prior distributions, Struct. Health Monit. 21 (2022) 1329–1344.
[25] D. Xu, M. Jia, T. Chen, Y. Liu, T. Chai, T. Yang, Decentralized federated domain generalization with cluster alignment for fault diagnosis, Control Eng. Pract. 148 (2024a) 105951.
[26] M. Russell, P. Wang, Maximizing model generalization for machine condition monitoring with self-supervised learning and federated learning, J. Manuf. Syst. 71 (2023) 274–285.
[27] W.J. Scheirer, A. De Rezende Rocha, A. Sapkota, T.E. Boult, Toward open set recognition, IEEE Trans. Pattern Anal. Mach. Intell. 35 (2013) 1757–1772.
[28] A. Bendale, T.E. Boult, Towards open set deep networks, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 1563–1572.
[29] C. Geng, S.J. Huang, S. Chen, Recent advances in open set recognition: A survey, IEEE Trans. Pattern Anal. Mach. Intell. 43 (2021) 3614–3631.
[30] Y. Chen, L. Tao, X. Liu, J. Ma, C. Lu, H. Liu, Open-set fault recognition and inference for rolling bearing based on open fault semantic subspace, IEEE Trans. Instrum. Meas. 73 (2024) 1–11.
[31] S. Lin, H. Zhao, Open-set fault diagnosis of wind turbine gearbox based on inverse lumped parameter thermal network and open-set kolmogorov–arnold network, IEEE Trans. Instrum. Meas. 74 (2025) 1–14.
[32] J. Li, R. Huang, G. He, Y. Liao, Z. Wang, W. Li, A two-stage transfer adversarial network for intelligent fault diagnosis of rotating machinery with multiple new faults, IEEE/ASME Trans. Mechatronics 26 (2021) 1591–1601.
[33] X. Yu, Z. Zhao, X. Zhang, Q. Zhang, Y. Liu, C. Sun, X. Chen, Deep-learning-based open set fault diagnosis by extreme value theory, IEEE Trans. Ind. Informatics 18 (2022) 185–196.
[34] H. Zhang, M. Cisse, Y.N. Dauphin, D. Lopez-Paz, Mixup: Beyond empirical risk minimization, 2017, arXiv preprint arXiv:1710.09412.
[35] P. Peng, J. Lu, T. Xie, S. Tao, H. Wang, H. Zhang, Open-set fault diagnosis via supervised contrastive learning with negative out-of-distribution data augmentation, IEEE Trans. Ind. Informatics 19 (2023) 2463–2473.
[36] J. Chen, G. Wang, J. Lv, Z. He, T. Yang, C. Tang, Open-set classification for signal diagnosis of machinery sensor in industrial environment, IEEE Trans. Ind. Informatics 19 (2023) 2574–2584.
[37] W.A. Smith, R.B. Randall, Rolling element bearing diagnostics using the case western reserve university data: A benchmark study, Mech. Syst. Signal Process. 64–65 (2015) 100–131.
[38] C. Lessmeier, J.K. Kimotho, D. Zimmer, W. Sextro, Condition monitoring of bearing damage in electromechanical drive systems by using motor current signals of electric motors: A benchmark data set for data-driven classification, in: Proceedings of PHM Society European Conference, 2016.
[39] P. Perera, V.I. Morariu, R. Jain, V. Manjunatha, C. Wigington, V. Ordonez, V.M. Patel, Generative-discriminative feature representations for open-set recognition, in: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 11811–11820.
[40] C. Yang, M. Zhu, Y. Liu, Y. Yuan, FedPD: Federated open set recognition with parameter disentanglement, in: 2023 IEEE/CVF International Conference on Computer Vision, 2023, pp. 4859–4868.
[41] L.D. Haan, A. Ferreira, Extreme Value Theory: An Introduction, Spring, USA, 2007.
[42] J. Davis, M. Goadrich, The relationship between precision-recall and roc curves, in: Proceedings of the 23rd International Conference on Machine Learning, 2006, pp. 233–240.
[43] A.R. Dhamija, M. Günther, T. Boult, Reducing network agnostophobia, in: Advances in Neural Information Processing Systems, 2018.
[44] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: Proceedings of International Conference on Learning Representations, 2015.
[45] L. Van der Maaten, G. Hinton, Visualizing data using t-SNE, J Mach. Learn. Res 9 (2008).